C Programming

Modularity & Data Structures

Modularity

- A module is a self-contained part of a system that has
  - A well-defined interface and
  - An implementation which hides the details and provides some functionalities

- Modularity helps to
  - Organize large programs in smaller parts (modules)
  - Make testable, understandable and reusable code
  - Provide clear and flexible interfaces
  - Have separate compilation and make changes easier
  - Hide implementation and support operations on data structure
  - Encapsulate, abstract data type and hide private data
    - An abstract data type is an incomplete data type (e.g. **FILE**)

Modularity

- Types of module
  - Single-instance module
    - A module that has a single set of data to manage
    - To encapsulate such a module, use **static** file scope objects
  - Multiple-instance module
    - A module that has to manage different sets of data for different clients
    - To encapsulate such a module, use **abstract data type**s and private data
      - Using **typedef** of a forward declared **struct** in the header (.h) file like

        typedef struct data data_t; // Forward declaration
      - In the source (.c) file, define the struct elements for **data_t**
      - Only **pointers** to an incomplete type can be defined and passed around.
        - It is not possible to dereference such a pointer.

Abstract data type

```
// module.h
#ifndef MODULE_H
#define MODULE_H

// Forward declaration
typedef struct data data_t;

#endif /* MODULE_H */
```

```
// module.c
#include "module.h"

struct data {
   int mem1;
   float mem2;
   /* ... */
};
```

Modularity

- To make an object in a module private, make it **static** in the .c file
- If it is required, make private data accessible through the module's public interface
  - By defining a set of function prototypes in the header (.h) file
- To make a module private in another module, include it in the .c file, otherwise in the .h file
- Use abstract data types to hide data. Memory for an ADT is allocated dynamically
  - Possible to convert a multiple-instance module to a single-instance module
    - If number of required instances is known
- Module **initialization** and **cleanup** functions
  - Every module that manages hidden data should have **initialization** and **cleanup** functions
    - E.g. a **create** and a **destroy** functions for each module
  - Modules that are made up of stand-alone functions, like strlen, sprintf, and etc. which keep no internal state won't need initialization and cleanup

Modularity

- Module Example

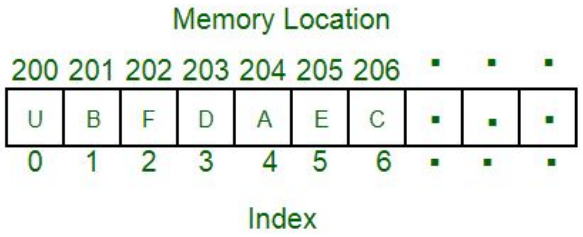
```c
#ifndef MODULE_H
#define MODULE_H

#ifdef __cplusplus
extern "C"
{
#endif
    /* ============================================================================ */
    /* ========================== include files =================================== */
    /* ============================================================================ */

    /* Inclusion of system and local header files goes here */

    /* ============================================================================ */
    /* ============================ constants ===================================== */
    /* ============================================================================ */

    /* #define and enum statements go here */

    /* ============================================================================ */
    /* ========================== public data ===================================== */
    /* ============================================================================ */

    /* Definition of public (external) data types go here */

    /* ============================================================================ */
    /* ======================= public functions =================================== */
    /* ============================================================================ */

    /* Function prototypes for public (external) functions go here */

#ifdef __cplusplus
}
#endif
#endif /* MODULE_H */
```

A typical source file of a module

```c
/* ============================================================================ */
/* ========================== include files =================================== */
/* ============================================================================ */

/* Inclusion of system and local header files goes here */

/* ============================================================================ */
/* ============================ constants ===================================== */
/* ============================================================================ */

/* #define and enum statements go here */

/* ============================================================================ */
/* ======================== global variables ================================== */
/* ============================================================================ */

/* Global variables definitions go here */

/* ============================================================================ */
/* ========================== private data ==================================== */
/* ============================================================================ */

/* Definition of private datatypes go here */

/* ============================================================================ */
/* ======================= private functions ================================== */
/* ============================================================================ */

/* Function prototypes for private (static) functions go here */

/* ============================================================================ */
/* ==================== All functions by section ============================== */
/* ============================================================================ */

/* Functions definitions go here, organised into sections */
```

Data Structure - Array

- Data structure: The way of organizing data for particular types of operation
- Some common data structures:
  - Array
  - Linked List
    - Singly linked list
    - Doubly linked list
  - Queue
    - Circular Buffer
  - Stack
  - Binary Tree



Array is a **fixed size** collection of items stored stored consecutively in a continuous piece of memory. Arrays allow random access of elements. Each element in an array can be accessed by its position in the array which is the **index** of the element.

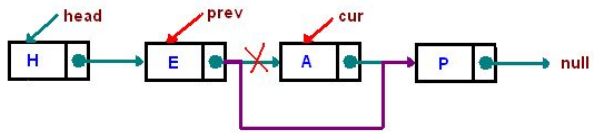
The index of the first element is 0 and the index of the last element is the number of the elements - 1.

Arrays have a better **cache locality** that can make a big difference in **performance**.

Data Structure - Linked List

- A linked list is a way to make a **dynamic list** as a series of connected nodes using pointers.

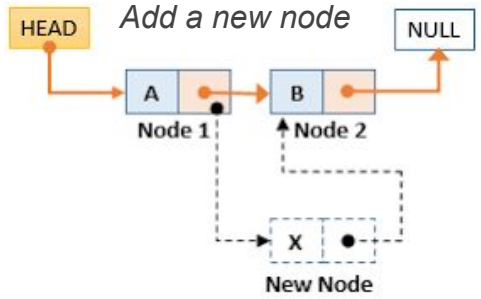
- The most common type is a **singly linked list**
  - Each node points to the next node



Delete a node (A) from a linked list

- We can dynamically add a new node
- We can delete any node in the list
- It does not allow random access to the nodes. To access a specific node we need to traverse the linked list usually starting from the **head** pointer which points to the first element.
- In a **doubly linked list** there is also a pointer to the previous node
  - This means we can traverse the list in any direction.

```c
typedef struct node
{
    uint8_t data;
    struct node *next;
} node_t;
```

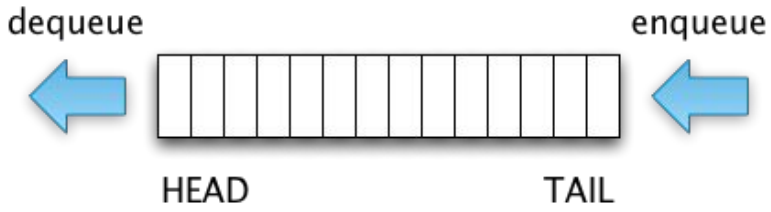



```c
typedef struct node
{
    uint8_t data;
    struct node *next;
    struct node *previous;
} node_t;
```

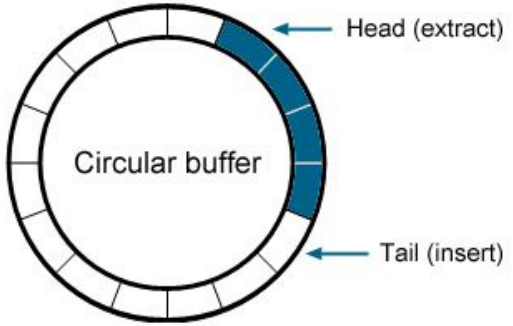

Akademin

Data Structure - Queue

- A queue is a linear data structure to store and retrieve data elements.
- It follows the order of First In First Out (FIFO).
- In a queue, the first entered element is the first one to be removed from the queue.
- Typical Operations Associated with a Queue
  - is_empty(): To check if the queue is empty or not
  - is_full(): To check whether the queue is full or not
  - dequeue(): Removes the element from the frontal side of the queue
  - enqueue(): It inserts elements to the end of the queue

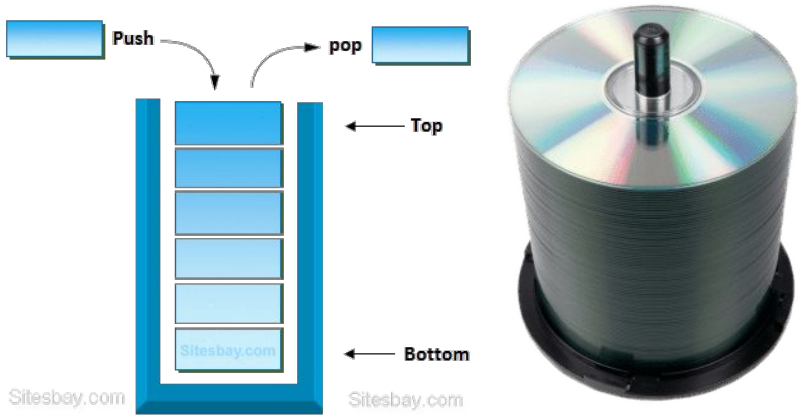


- A queue can be implemented using
  - An array - It is called a ring or circular queue/buffer.
  - A linked list.



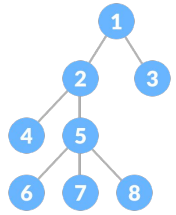
Data Structure - Stack

- A stack is a linear data structure to store and retrieve the data elements.
- It follows the order of Last In First Out (LIFO).
- In a stack, the first entered element is the last one to be retrieved from the stack.
- Typical Operations Associated with a Stack
  - is_empty(): To check if the stack is empty or not
  - is_full(): To check if the stack is full or not
  - pop(): Removes an item from the top of the stack.
    - If the stack is empty we have an underflow condition
  - push(): Inserts an item in the stack.
    - If the stack is full we have an overflow condition.
- A stack can be implemented using an array or a linked list

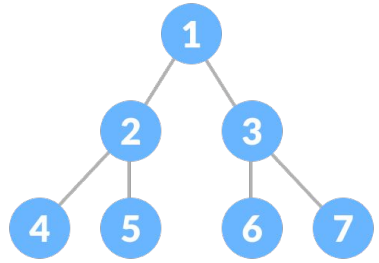


Data Structure - Binary Tree

- A tree is a nonlinear hierarchical data structure that consists of nodes connected by edges (pointers)

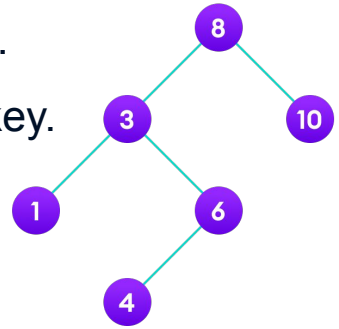


- A binary tree is a tree data structure in which each parent node can have at most two children.




```
typedef struct node
{
    struct node *right;
    struct node *left;
    int data;
} node_t;
```

- Binary Search Tree (**BST**) is a binary tree data structure which has the following properties:
  - The left subtree of a node contains only nodes with keys lesser than the node's key.
  - The right subtree of a node contains only nodes with keys greater than the node's key.
  - The left and right subtrees each must also be a binary search tree.



- Typical operations on a BST: insert, edit, delete, search and traverse
- A BST is automatically sorted. It provides quicker access/search than linked lists

Data Structure

- Some useful links
  - Linked List Data Structure
  - Linked List Tutorial
  - C Linked List
  - Stack Data Structure
  - Queue Data Structure
  - Circular Buffer Structure
  - Ring Buffer (Circular Buffer)
  - Binary Tree Data Structure
  - Binary Search Tree (BST)
  - Data Structures Easy to Advanced Course